

Exercise Sheet 4

Linear Models: Least Squares, Logistic Regression and Calibrated Scores

Probabilistic Machine Learning

Prof. Dr. Michael Granitzer

4.A Least Squares Is a Likelihood

Exercise 4.A.1: Four Flats and a Villa

Model $y = w_0 + w_1x + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ i.i.d.; x in m², y in k€.

x	40	60	80	100
y	150	200	280	330

1. Write down $\ell(w_0, w_1, \sigma^2)$ and show that for **every** σ^2 the maximum over (w_0, w_1) is the least-squares fit.
2. Compute $\hat{w}_1 = S_{xy}/S_{xx}$ and $\hat{w}_0 = \bar{y} - \hat{w}_1\bar{x}$, the residuals and the RSS.
3. Compute $\hat{\sigma}_{\text{ML}}^2 = \text{RSS}/n$ and the unbiased $\text{RSS}/(n - 2)$. By which factor does the ML version underestimate σ^2 on average?
4. Give the plug-in 95% band $\hat{y} \pm 1.96\hat{\sigma}_{\text{ML}}$ at 80 m² and at 600 m². Then use the half-width that counts the uncertainty about $\mathbf{w}$, $1.96\hat{\sigma}\sqrt{1 + \frac{1}{n} + \frac{(x-\bar{x})^2}{S_{xx}}}$. What do you conclude?

Exercise 4.A.1: Four Flats and a Villa — Solution

Exercise 4.A.1: Four Flats and a Villa — Solution

1. $\ell = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_i (y_i - w_0 - w_1 x_i)^2$. The first term has no w , and $\frac{1}{2\sigma^2} > 0$, so the arg max over w is the arg min of the RSS, whatever σ^2 is.
2. $\bar{x} = 70, \bar{y} = 240, S_{xx} = 2000, S_{xy} = 6200$: $\hat{w}_1 = 3.1$ k€ per m², $\hat{w}_0 = 240 - 217 = 23$. Fitted 147, 209, 271, 333; residuals 3, -9, 9, -3; RSS = 180.
3. $\hat{\sigma}_{\text{ML}}^2 = 45$ ($\hat{\sigma} \approx 6.7$); RSS/2 = 90 (≈ 9.5). Since $E[\text{RSS}] = (n - 2)\sigma^2$, the ML version is on average $\frac{n-2}{n} = \frac{1}{2}$ of σ^2 — with 4 points and 2 weights, half the noise disappears into the fit.
4. Plug-in: 271 ± 13 at 80 m², 1883 ± 13 at 600 m². With the uncertainty about $\mathbf{w}$: factor $\sqrt{1.3} \approx 1.14$ at 80 m² (± 15), $\sqrt{141.7} \approx 11.9$ at 600 m² (± 157). The plug-in band is as narrow at the villa as at a flat; the honest one is twelve times wider — and even that one trusts the straight line out to 600 m².

4.B Classification: Only $p(y | x)$

Exercise 4.B.1: Cross-Entropy and Its Gradient

$h_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$ with $\sigma(z) = 1/(1 + e^{-z})$ and $\ell(\mathbf{w}) = \sum_i [y_i \ln h_i + (1 - y_i) \ln(1 - h_i)]$.

1. Show $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
2. Show $\frac{\partial \ell}{\partial \mathbf{w}} = \sum_i (y_i - h_i) \mathbf{x}_i$.
3. Six transactions, $\mathbf{x} = (1, \text{new device})$: new device — fraud, fraud, legitimate; known device — fraud, legitimate, legitimate. Compute the gradient at $\mathbf{w} = 0$.
4. Set both components of the gradient to zero: which scores does the MLE give to new and to known devices? Give $\hat{w}_0, \hat{w}_1$ and read $\hat{w}_1$ as an odds ratio.

Exercise 4.B.1: Cross-Entropy and Its Gradient — Solution

Exercise 4.B.1: Cross-Entropy and Its Gradient — Solution

1. $\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)(1 - \sigma(z)).$
2. $\frac{\partial}{\partial \mathbf{w}} [y \ln h + (1 - y) \ln(1 - h)] = \left(\frac{y}{h} - \frac{1-y}{1-h}\right) h(1 - h)\mathbf{x} = (y - h)\mathbf{x}$; sum over i . Error $\times$ feature, as in LMS.
3. At $\mathbf{w} = 0$ every $h_i = 0.5$. Intercept: $\sum_i (y_i - 0.5) = 3 - 3 = 0$. Slope: only new devices count, $0.5 + 0.5 - 0.5 = 0.5$. Gradient $(0, 0.5)$: raise the weight of "new device".
4. Slope component: $\sum_{\text{new}} (y_i - h_{\text{new}}) = 2 - 3h_{\text{new}} = 0$, so $h_{\text{new}} = 2/3$; then the intercept component gives $h_{\text{known}} = 1/3$ — each group's observed fraud rate. $\hat{w}_0 = \ln \frac{1/3}{2/3} = \ln \frac{1}{2} \approx -0.69$, $\hat{w}_1 = \ln 2 - \ln \frac{1}{2} = \ln 4 \approx 1.39$: a new device multiplies the odds of fraud by 4.

Exercise 4.B.2: The Peak That Runs Away (Python)

```
import numpy as np
x = np.array([-2.0, -1.5, -1.1, -0.6, -0.3, 0.2, 0.6, 1.0, 1.4, 1.9])
y = np.array([0, 0, 0, 0, 0, 0, 1, 1, 1, 1])
sigma = lambda z: 1 / (1 + np.exp(-z))

def fit(tau2=None, steps=10_000, eta=0.05):
    w0 = w1 = 0.0
    for t in range(1, steps + 1):
        h = sigma(w0 + w1 * x)
        g0 = ___ # gradient for the intercept
        g1 = ___ # gradient for the slope
        if tau2 is not None:
            g1 -= ___ # log-prior N(0, tau2) on the slope
        w0, w1 = w0 + eta * g0, w1 + eta * g1
        if t in (10, 100, 1_000, 10_000):
            h = sigma(w0 + w1 * x)
            print(t, round(w1, 2), np.sum(y * np.log(h) + (1 - y) * np.log(1 - h)))
```

1. Fill in the blanks and run `fit()`. What happens to the slope, the log-likelihood and the scores? Why will it never stop?
2. Run `fit(tau2=1)` and `fit(tau2=100, steps=100_000)`. Explain the difference with the MAP formula of the lecture.

Exercise 4.B.2: The Peak That Runs Away — Solution

Exercise 4.B.2: The Peak That Runs Away — Solution

```
g0 = np.sum(y - h)
g1 = np.sum((y - h) * x)
g1 -= w1 / tau2
```

1. Slope $1.37 \rightarrow 3.64 \rightarrow 8.32 \rightarrow 17.76$; log-likelihood $-2.68 \rightarrow -1.04 \rightarrow -0.36 \rightarrow -0.06$. The data are separable, so scaling up the weights always raises the likelihood towards 0: the scores approach exactly 0 and 1 and the peak does not exist — three heads in three flips.
2. $\tau^2 = 1$: slope 1.43 from about 100 steps on, log-likelihood -2.54 . The log-prior $-w_1^2/(2\tau^2)$ outgrows the vanishing gain in likelihood, so a finite MAP exists. $\tau^2 = 100$: a weaker prior, slope 7.83 — finite, but steep. $\lambda = 1/\tau^2$ decides how far the peak is allowed to run.

4.C Generative vs Discriminative

Exercise 4.C.1: The Same Line

Naive Bayes with yes/no flags, $\theta_{jy} = P(x_j = 1 \mid y)$, prior $\pi = P(y = 1)$. Lecture numbers: $\pi = 1/1000$; new device 0.6 vs 0.1; foreign IP 0.7 vs 0.05; night 0.5 vs 0.2 (fraud vs honest).

1. Show that $\log \frac{P(y=1|x)}{P(y=0|x)} = w_0 + \sum_j w_j x_j$ and give w_j and w_0 .
2. Compute the three weights and w_0 , and check that a transaction with all three flags gets $P(\text{fraud} \mid x) \approx 17.4\%$.
3. The foreign-IP flag is **missing**. Compare what naive Bayes should do with plugging $x_{\text{IP}} = 0$ into the linear form.
4. Why can logistic regression, which has the same linear form, not do what naive Bayes does in 3?

Exercise 4.C.1: The Same Line — Solution

Exercise 4.C.1: The Same Line — Solution

1. $\log p(x_j \mid y) = x_j \log \theta_{jy} + (1 - x_j) \log(1 - \theta_{jy})$. Take the difference between the classes and sum over j :

$$w_j = \log \frac{\theta_{j1}}{\theta_{j0}} - \log \frac{1 - \theta_{j1}}{1 - \theta_{j0}}, \quad w_0 = \log \frac{\pi}{1 - \pi} + \sum_j \log \frac{1 - \theta_{j1}}{1 - \theta_{j0}}$$

2. $w_{\text{dev}} = \log 6 - \log \frac{0.4}{0.9} = 1.792 + 0.811 = 2.603$; $w_{\text{IP}} = \log 14 - \log \frac{0.3}{0.95} = 2.639 + 1.153 = 3.792$;
 $w_{\text{night}} = \log 2.5 - \log \frac{0.5}{0.8} = 0.916 + 0.470 = 1.386$. $w_0 = \log \frac{1}{999} - 0.811 - 1.153 - 0.470 = -9.340$.
. All flags: $-9.340 + 7.781 = -1.560$, $\sigma(-1.560) = 0.174$ ✓ — the same 210/1,209 as in the lecture.

3. Missing: drop the IP factor entirely (it sums to 1 over both values), odds $6 \cdot 2.5/999 = 15/999$, $P \approx 1.5\%$.
Plugging in $x_{\text{IP}} = 0$ instead claims "domestic IP observed": it keeps the factor $0.3/0.95$ (hidden in w_0), odds $4.74/999$, $P \approx 0.47\%$ — a third of the honest answer.

4. Logistic regression models only $p(y \mid x)$; it has no $p(x_{\text{IP}} \mid y)$ to sum over. Its w_0 is not split into per-attribute pieces, so there is nothing to remove — it must be given some value of x_{IP} or be refitted without the feature.

Exercise 4.C.2: Counting Twice, in Code

Complete the three marked lines (numpy only), then run the script.

```
import numpy as np
rng = np.random.default_rng(42)
n = 5000
y = (rng.random(n) < 0.3).astype(int)          # 1 = fraud
flag = lambda pf, ph: (rng.random(n) < np.where(y == 1, pf, ph)).astype(int)
device = flag(0.6, 0.1)
copy = np.where(rng.random(n) < 0.95, device, 1 - device)  # a near-copy
night = flag(0.5, 0.2)
X = np.column_stack([device, copy, night])

def fit_nb(X, y, alpha=1.0):
    prior = ...                # (a)  $P(y = 1)$ 
    th1 = ...                   # (b) Laplace-smoothed  $P(x_j = 1 \mid y = 1)$ 
    th0 = ...                   # and  $P(x_j = 1 \mid y = 0)$ 
    w = np.log(th1 / th0) - np.log((1 - th1) / (1 - th0))
    w0 = np.log(prior / (1 - prior)) + np.log((1 - th1) / (1 - th0)).sum()
    return w0, w

def fit_lr(X, y, steps=5000, lr=0.5):  # gradient ascent, as in the lecture
    Xb = np.column_stack([np.ones(len(X)), X]); w = np.zeros(Xb.shape[1])
    for i in range(steps):
```

1. Print w_0 , w and $P(\text{fraud} \mid x = (1, 1, 1))$ for both models.
2. Compare the weights of device and copy. Which model counts the device twice, and which shares the credit?
3. Which probability is closer to the fraction of frauds among the training rows with $x = (1, 1, 1)$?

Exercise 4.C.2: Counting Twice, in Code — Solution

Exercise 4.C.2: Counting Twice, in Code — Solution

```
prior = y.mean()
th1 = (X[y == 1].sum(axis=0) + alpha) / ((y == 1).sum() + 2 * alpha)
th0 = (X[y == 0].sum(axis=0) + alpha) / ((y == 0).sum() + 2 * alpha)
```

1. (numpy 2.x, seed 42)

model	w_0	w_{device}	w_{copy}	w_{night}	$P(\text{fraud} \mid 1, 1, 1)$
naive Bayes	-2.80	2.60	2.17	1.35	0.965
logistic regression	-2.10	2.75	-0.13	1.40	0.873

2. Naive Bayes gives the copy almost the device's weight: the same evidence enters twice. Logistic regression fits all weights jointly, sees that the copy adds nothing once the device is known, and drives its weight to about 0.

3. Among the 486 training rows with $x = (1, 1, 1)$, 88.5% are fraud; the true posterior of the generating model is 0.865. Logistic regression (0.873) is close; naive Bayes (0.965) is **over-confident** — the double counting of Lecture 03, measured.

4.D When Is a Score a Probability?

Exercise 4.D.1: The Rebalanced Fraud Model

A model is trained on a 50/50 sample; on real traffic fraud is **1 in 1,000**. Undersampling kept every fraud and a random share of legitimate cases.

1. Use Bayes' theorem in odds form to show: $\text{real odds} = \text{model odds} \times \frac{1/999}{1}$. Which assumption about $p(\mathbf{x} \mid y)$ do you need?
2. Convert the scores 0.5, 0.9 and 0.99 into real fraud probabilities.
3. Blocking a legitimate card costs €10, missing a fraud €500. Above which real probability should a card be blocked, and which **raw score** is that?

Exercise 4.D.1: The Rebalanced Fraud Model — Solution

Exercise 4.D.1: The Rebalanced Fraud Model — Solution

1. Posterior odds = $\frac{p(\mathbf{x}|\text{fraud})}{p(\mathbf{x}|\text{legit})} \times \text{prior odds}$. Sampling that depends only on y leaves $p(\mathbf{x} \mid y)$ unchanged, so the likelihood ratio is the same in both worlds; only the prior odds change, from 1 (training) to $1/999$ (real).
2. 0.5: odds $1/999$, $p = 0.10\%$. 0.9: odds $9/999$, $p = 9/1008 \approx 0.89\%$. 0.99: odds $99/999$, $p = 99/1098 \approx 9.0\%$.
3. Block if $500p > 10(1 - p)$, i.e. $p > \frac{10}{510} \approx 1.96\%$ (real odds > 0.02). Raw odds $> 0.02 \cdot 999 = 19.98$, raw score $> 19.98/20.98 \approx 0.95$. A threshold of 0.5 on the raw score would block cards whose real risk is 0.1%.

Exercise 4.D.2: Weight Decay Is a Prior

1. With $y_i = \mathbf{w}^\top \mathbf{x}_i + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ and a prior $\mathbf{w} \sim \mathcal{N}(0, \tau^2 \mathbf{I})$, show that the MAP minimises $\text{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|^2$ with $\lambda = \sigma^2 / \tau^2$.
2. One weight, no intercept, $y = wx + \varepsilon$. Show $\hat{w}_{\text{MAP}} = \frac{\sum_i x_i y_i}{\sum_i x_i^2 + \lambda}$.
3. Data $(x, y) = (1, 2), (3, 7)$, $\sigma^2 = 1$, $\tau^2 = 0.1$. Compute the MLE and the MAP. What happens as $\tau^2 \rightarrow \infty$ and as $\tau^2 \rightarrow 0$?

Exercise 4.D.2: Weight Decay Is a Prior — Solution

Exercise 4.D.2: Weight Decay Is a Prior — Solution

1. $\log p(\mathbf{w} \mid D) = -\frac{\text{RSS}(\mathbf{w})}{2\sigma^2} - \frac{\|\mathbf{w}\|^2}{2\tau^2} + \text{const.}$ Multiplying by $-2\sigma^2$ turns the maximum into the minimum of $\text{RSS} + \frac{\sigma^2}{\tau^2} \|\mathbf{w}\|^2$.
2. The derivative of $\sum_i (y_i - wx_i)^2 + \lambda w^2$ is $-2 \sum_i x_i y_i + 2w \sum_i x_i^2 + 2\lambda w$; zero at the formula.
3. $\sum x_i y_i = 23$, $\sum x_i^2 = 10$, $\lambda = 10$. MLE $23/10 = 2.3$; MAP $23/20 = 1.15$. $\tau^2 \rightarrow \infty$: $\lambda \rightarrow 0$, the MLE (a flat prior). $\tau^2 \rightarrow 0$: $\lambda \rightarrow \infty$, $\hat{w} \rightarrow 0$ — the prior overrules two data points, as a strong Beta prior did in Lecture 02.